

CSCE 625 ARTIFICIAL INTELLIGENCE · MODULE 04 · LECTURE NOTES

Pathfinding in Practice

The search that ships in every engine.

MODULE OUTCOMES

- Choose a spatial representation and justify it.
- Explain navigation meshes and how they are built.
- Implement hierarchical and any-angle pathfinding.
- Handle dynamic obstacles and replanning under a frame budget.
- Separate pathfinding from path following.

1 Spatial representation

Representation	Nodes are	Trade
Grid	One per cell.	Trivial to build from a tilemap; huge node count; paths look blocky (§2). Fine for tile-based games, poor for continuous worlds.
Waypoint graph	Hand-placed points with edges.	Tiny and extremely fast; requires manual placement; agents visibly hug the waypoints rather than using the free space, which was the characteristic look of late-1990s game AI.
Navigation mesh	Convex polygons covering the walkable floor.	The standard answer. Few nodes, agents use the full free space, and any-angle paths come naturally (§2).
Visibility graph	Obstacle corners.	Yields exactly optimal 2D paths (the optimal path bends only at corners), and costs $O(n^2)$ to build — CSCE 620 Module 06's structure, used here.
Probabilistic roadmap / RRT	Random configuration samples joined by a local planner.	For high-degree-of-freedom configuration spaces — a robot arm, a vehicle with a turn radius — not for walking on floors. See §4.

The navigation mesh won because its node count scales with the *complexity* of the level rather than with its area. A large empty room is one polygon and ten thousand grid cells — and since Module 03 §2 established that search cost is exponential in depth, reducing the node count by three orders of magnitude is not a constant-factor saving.

A NAVIGATION MESH IS A CONVEX DECOMPOSITION OF WALKABLE FLOOR

Partition the walkable surface into convex polygons, marked adjacent where an agent can cross directly from one to the next. **Search the polygon adjacency graph with A***, exactly as in Module 03.

Convexity is the entire point. Inside a single convex polygon, straight-line movement between any two points is guaranteed collision-free — so the search only needs to find the *sequence* of polygons, and the geometry within each is solved (which is what makes §2's funnel algorithm exact).

Built by voxelising the level geometry, extracting the walkable surface by slope and height clearance, then region-growing the voxel surface into areas and polygonising those — which is what Recast does, and it runs offline per level as part of the content pipeline. **CSCE 645's surface representations and CSCE 620's polygon decomposition both appear here**, which is why those courses preceded this one.

Agent radius is baked in by eroding the walkable area by that radius, so a navigation mesh is specific to one agent size. Which is why engines build several — one per size class — and **why a large enemy sometimes cannot follow you through a gap it visually appears to fit**: it is using a different, more eroded mesh.

2 Any-angle paths

A* ON A GRID gives a path restricted to grid edges, so it zigzags along 45-degree diagonals where a straight line was available. It is OPTIMAL ON THE GRAPH and wrong for the world -- because the graph was your approximation of the world (Module 01 section 2).

THREE FIXES:

STRING PULLING / THE FUNNEL ALGORITHM

take the polygon sequence from the navmesh search and pull the path taut within that corridor. $O(n)$ in the corridor length and EXACT within the corridor. The standard answer for navigation meshes.

THETA* / LAZY THETA*

during the search, allow a node's parent to be any ancestor with line of sight rather than only the immediately preceding node. Produces any-angle paths directly, at the cost of line-of-sight checks during search.

POST-HOC SMOOTHING

repeatedly attempt to replace two consecutive segments with one, where line of sight permits. Simple, and it can cut corners the agent cannot physically take, so it needs a clearance check.

THE CAVEAT ON STRING PULLING: it is optimal WITHIN THE CORRIDOR the search found, and that corridor need not contain the globally optimal path -- because the search optimised polygon-sequence cost, not Euclidean length. Usually immaterial, occasionally visible.

3 Scaling to many agents and large levels

- **Hierarchy.** Partition the level into clusters, precompute paths between cluster entrances, search the cluster graph, then refine within clusters. **HPA* reduces node counts by orders of magnitude for paths within a few percent of optimal** — Module 02 §4's abstraction, with numbers.
- **Precomputation.** All-pairs distances between cluster entrances, or a full flow field where many agents share one destination — justified by Module 03 §2's amortisation argument, since a level is searched thousands of times.
- **Flow fields.** One Dijkstra outward from the goal labels every node with its direction-to-goal, and then every agent simply reads its cell. The cost is independent of the number of agents, which is how real-time strategy games move hundreds of units to one destination. **The key insight is to invert the problem:** one search from the goal rather than one search per agent.
- **Time-slicing.** Spread a single search across several frames, budgeting a fixed number of node expansions per frame and returning the result when it completes. Requires the search state to be resumable, and requires the game to tolerate a few frames of latency — which it almost always does.
- **And caching.** Agents repeat queries constantly, especially to common destinations. **A cache keyed on (start polygon, goal polygon) pays immediately** and needs invalidation only when the mesh changes.

DYNAMIC OBSTACLES, AND WHEN TO REPLAN

A door closes, a crate is pushed into a corridor, another agent stops in a doorway. The path you computed three seconds ago is now invalid, and the agent does not know.

Full replanning every frame is unaffordable at any realistic agent count, and replanning only when a collision occurs is too late — the agent is already pressed against the obstacle, which is exactly the visible failure.

D* Lite repairs the previous search result rather than redoing it, at a cost proportional to how much of the world changed rather than to the level size. The right tool for a mostly-static world with local changes, which is what a game level is.

And local avoidance is a separate layer entirely. Agents avoid each other by steering, not by replanning (Module 11 §3), because the path is a plan and avoidance is a reflex — Module 01 §1's agent ladder, with the cheapest design handling the fast-changing part and the expensive one handling the slow-changing part. Getting that division wrong is the most common architectural error in game navigation.

4 Path following, and debugging

PATHFINDING AND PATH FOLLOWING ARE DIFFERENT PROBLEMS

Pathfinding answers 'which way'. **Path following** answers 'how to move along it' — subject to acceleration limits, turn radius, animation state, foot placement, and the presence of other agents.

Conflating them means searching over a far larger state space — position *and* velocity *and* heading, which is Module 02 §1's state-choice decision made expensively — **for very little benefit in a game where characters can turn in place.**

So the standard architecture is three layers: a global path from the navigation mesh (slow, replanned rarely), local steering along it (fast, every frame), and collision resolution underneath (every frame, geometric). **Each layer handles a different timescale, which is why the decomposition works.**

And when the vehicle's constraints genuinely matter — a car, a tank with a minimum turn radius, a robot arm — **you need kinodynamic planning, which is a different and substantially harder problem** and is where §1's RRT and probabilistic roadmaps belong. **Know which case you are in before choosing a method,** because the floor-walking case is far easier and the methods do not transfer.

- **The agent walks into a wall. The navigation mesh does not match the collision geometry** — the mesh says walkable, physics says solid. **The most common bug in practice, and it is a content problem rather than a code problem,** usually caused by geometry edited after the mesh was baked.
- **The agent takes an obviously long way round.** A missing mesh connection (a jump link, a ladder), or an edge cost that does not reflect designer intent — both visible by rendering the mesh.
- **The agent oscillates between two paths. Replanning is flip-flopping between two near-equal options** as conditions change slightly. **Add hysteresis:** require the new path to be better by a margin before switching.
- **Agents bunch up at a doorway.** They all share one optimal path and local avoidance is fighting the path. **This is Module 11 §3's problem,** and the fixes are path variation, queueing behaviour, or a crowd model rather than a pathfinding change.
- **Frame-time spikes.** An unbounded search ran to completion inside one frame. **Cap expansions per frame, and measure the 99th percentile rather than the mean** — CSCE 678 Module 06's lesson, which is what a frame budget actually requires.
- **Render the navigation mesh and the computed path inside the running game. Almost every pathfinding bug is immediately visible this way and effectively invisible otherwise** — the same 'look at the frame rather than the numbers' discipline as CSCE 647 and CSCE 753 Module 05.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Recast & Detour (free, open source)	The navigation mesh builder of §1 , used in many shipped games. Readable, and worth reading for the voxelisation pipeline.
Botea, Müller & Schaeffer — Near Optimal Hierarchical Path-Finding (HPA*) (free)	The §3 hierarchy , with the node-count reductions measured.
Koenig & Likhachev — D* Lite (free)	The §3 replanning method , and notably simpler than the original D*.
Red Blob Games — A* and pathfinding articles (free, interactive)	The best practical treatment of this module available , including grids, navmeshes, smoothing, and flow fields, all interactive.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Implement grid A* and render the path** in a level. Observe the zigzag.
- 2 **Implement the funnel algorithm** on a navigation mesh corridor and compare path lengths.
- 3 **Implement Theta*** and compare its paths and node counts against smoothed A*.
- 4 **Build a navigation mesh** for a level, by hand or with Recast, and report the node count against the grid's.
- 5 **Implement HPA*** and report nodes expanded and path suboptimality against plain A*.
- 6 **Implement a flow field** and move 500 agents to one goal. Report the cost against 500 individual searches.
- 7 **Time-slice a search** across frames with a fixed expansion budget and verify the frame time stays bounded.
- 8 **Close a door mid-path** and compare full replanning against D* Lite in node counts.
- 9 **Create the oscillation bug deliberately** and fix it with hysteresis.
- 10 **Measure p50 and p99 pathfinding time** with 100 agents and report both.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Compare five spatial representations and say why navmeshes won.
- 2 Why is convexity the essential property of a navmesh polygon?
- 3 How is a navmesh built, and why is it per-agent-size?
- 4 Why does grid A* zigzag, and what are three fixes?
- 5 What is the caveat on string pulling?
- 6 Name five scaling techniques and what each exploits.

- 7 Why are flow fields the answer for crowds?
- 8 When should you replan, and what does D* Lite do?
- 9 Why separate pathfinding from path following?
- 10 Give five pathfinding bugs and the cause of each.